

Lab 2 Report

CSE 60685 Machine Learning for Embedded Systems LeNet Architecture Comparison on Raspberry Pi

Device and settings. Supplied Raspberry Pi 4B Rev. 1.5; Raspberry Pi OS 64-bit (Debian 13), Python 3.13.5, PyTorch 2.8.0+cpu. All training, evaluation and timing ran on the Pi CPU with one intra-op and one inter-op thread. Same CanaKit power supply, closed case, running fan and ondemand governor; sequential runs. Pre-warmup benchmark temperatures were 42.8–44.3 °C; all recorded power/throttling flags were 0x0.

Common recipe. Fashion-MNIST: supplied 12,000/2,000 training/validation split and all 10,000 official test images; grayscale 28×28 float32 inputs in $[0, 1]$, no extra normalization or augmentation. Each design was fixed before test evaluation and trained from scratch: 5 epochs, batch 64, Adam, learning rate 0.001, seed 42, zero loader workers. Final-epoch checkpoints were reloaded and verified; A also passed a separate verify-only process before test evaluation. C uses hidden widths **60 and 42**.

Model	Parameters	Test accuracy (%)	Mean training (s/epoch)	Median inference (ms)
A: baseline	44,426	79.72	13.86	2.6603
B: conv-small	27,180	78.05	10.17	2.3970
C: FC-small	20,984	78.84	13.56	2.4440

Timing. Unchanged instructor `train.py`, `evaluate.py`, `benchmark.py` and `summarize.py`. Training time is the mean of all five training loops, including data iteration, forward/backward, updates and fixed statistics; validation, test and output are excluded. After all training, inference used the same prepared $[1, 1, 28, 28]$ tensor, 10 untimed warmups and 100 timed calls per model in evaluation/inference mode. The timer covers synchronous `model(tensor)` plus Python call overhead, excluding loading, preprocessing, `argmax/softmax` and output.

1. Architecture changes

A uses convolution channels 6/16 and hidden widths 120/84. B changes the channels to 3/8, retaining 120/84. For C, I chose to halve both hidden widths to 60/42 while keeping channels 6/16. The second pooling layer produces $c_2 \times 4 \times 4$ features, so the first Linear layer needs $16c_2$ inputs: 256 for A/C and 128 for B. Kernels, pooling and activations are unchanged.

2. Parameter count and measured speed

C has the largest parameter reduction: **52.77%**, versus 38.82% for B. B has the largest measured inference speedup: **1.11×** versus A, compared with 1.09× for C (baseline median divided by candidate median). This supports my interpretation that architecture matters, not just parameter count. C removes weights from the final classifier but keeps the convolutional feature maps and their repeated spatial computation. B reduces those channels as well as the first Linear input. In this run, B also trained faster (10.17 versus 13.86 s/epoch); C's training time remained closer to A (13.56 s/epoch). Aggregate timings do not isolate each layer's contribution.

3. Model choice

I would choose **C** because my priority is fewer parameters and the accuracy differences here are small enough for that tradeoff. C uses 20,984 parameters, with test accuracy 0.88 percentage points below A and 0.79 points above B. Its inference median is lower than A's and only 0.0470 ms higher than B's. Thus, I prefer C's smaller weight-storage requirement while accepting this measured accuracy/runtime tradeoff. These are results from one training run per design, not evidence of statistical equivalence.

AI Attribution Appendix

Lab 2 — excluded from the one-page analysis limit

Tool and scope

OpenAI Codex assisted with understanding the course materials, explaining hardware steps in Chinese, setting up the Pi environment through SSH, transcribing supplied tutorial code, running commands, checking evidence, explaining the comparisons, translating/editing my interpretation, and LaTeX formatting. The OMEN laptop was the control and typesetting machine; the reported training, accuracy and inference measurements all came from the actual supplied Raspberry Pi.

Prompts and student decisions

The following summarizes the relevant prompts and responses (Chinese messages are translated into English here; the detailed session record is retained):

1. “Lab 2 has been released; explain it step by step and help me complete it.” I subsequently requested Chinese explanations.
2. I reported the card/power state and confirmed the measurement cooling condition: case closed, fan running.
3. For model C, I chose “60 and 42 — halve both layers” after the baseline sizes and smaller-width options were explained.
4. “Are our current A, B and C models all compliant with Lab 2?” Codex compared the configurations with the assignment and inspected the three passing checks on the Pi.
5. After seeing the measured comparison, I was asked which model I would choose and how I understood C having fewer parameters while B was faster. My response was: “I think the model with the fewest parameters is best because the accuracies are all quite similar. C has few parameters, but B is faster; this is related to the architecture.” I then explained that C keeps convolutional computation and shrinks the final fully connected part. Codex clarified that both hidden widths change (120/84 to 60/42), with ten output classes unchanged. These responses formed the basis of the discussion; Codex translated/edited them and made the numerical tradeoffs explicit.

Code and operational assistance

The completed `LeNet.forward` and `train_step` function bodies were transcribed from the instructor’s Tutorial Steps 2 and 3. They were supplied course code, rather than a newly generated AI model implementation. B’s dictionary uses the prescribed 3/8 channels and 120/84 hidden widths; C’s dictionary uses my chosen 6/16 channels and 60/42 hidden widths. Among instructor files, only `models.py` and `train_step.py` changed.

Codex wrote an external operational helper to invoke the unchanged instructor programs sequentially, record commands and telemetry, preserve output logs and wait for cooling. It also wrote an export audit and the LaTeX generation support. These helpers are disclosed as AI-written assistance; they are separate from the course model and timing implementation. No reported values were generated on OMEN, invented or substituted for Pi measurements.

Validation and limitations

The Pi environment and all three model checks passed. Every design completed five epochs, checkpoint reload verification, all 10,000 test images and the required 10 warmups/100 timed inference calls. The instructor’s summarizer verified provenance and recomputed statistics. A separate audit passed 144 source, transfer and measurement checks, including hashes and comparisons with the raw CSV files. All observations are retained; no timed samples were discarded. The experiment used one fixed seed and one trained checkpoint per model, so the report treats the results as this measured comparison, without claiming significance across repeated training runs.